

VidSumEval: A Web Platform and Benchmark for Evaluating AI-Generated Programming Video Summaries

Osama AlGhamdi
University of Jeddah
Jeddah, Saudi Arabia
2340546@uj.edu.sa

Ameer AlGhamdi
University of Jeddah
Jeddah, Saudi Arabia
2340533@uj.edu.sa

Faisal Alorabi
University of Jeddah
Jeddah, Saudi Arabia
2341120@uj.edu.sa

Mohammad D. Alahmadi
University of Jeddah
Jeddah, Saudi Arabia
mdalahmadi@uj.edu.sa

Abdulmalik Alzahrani
University of Jeddah
Jeddah, Saudi Arabia
2340534@uj.edu.sa

Abdullah Almazroui
University of Jeddah
Jeddah, Saudi Arabia
2340644@uj.edu.sa

Abstract

Programming tutorial videos on YouTube are a widely used learning resource, but their length often makes content review inefficient. AI-based summarization tools can produce shorter versions of these videos, yet existing evaluation relies almost exclusively on similarity-based metrics that measure surface overlap rather than whether the summaries actually support learning. We present VidSumEval, a web platform and benchmark for evaluating AI-generated programming tutorial summaries from a learning-centered perspective. The benchmark pairs 10 tutorials in C, Java, and Python with summaries from two commercial engines (VEED and NotebookLM) and auto-generated comprehension quizzes; users can also submit new YouTube tutorials for offline review. We validate the benchmark with a within-subject study (n=20) measuring both quiz-based comprehension and Likert-scale ratings on completeness, clarity, and coherence; learning effectiveness is reported as the per-participant difference between summarized- and original-video quiz scores. The platform is available at https://progsumeval.onrender.com; the source code and dataset are archived at https://doi.org/10.5281/zenodo.21763079; and the screencast is available at https://youtu.be/iBB-5HfRVSk.

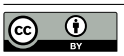
CCS Concepts

• **Applied computing** → **Computer-assisted instruction**.

Keywords

Video summarization, programming education, benchmark, large language models, learning effectiveness

ACM Reference Format:
Osama AlGhamdi, Ameer AlGhamdi, Faisal Alorabi, Mohammad D. Alahmadi, Abdulmalik Alzahrani, and Abdullah Almazroui. 2026. VidSumEval: A Web Platform and Benchmark for Evaluating AI-Generated Programming Video Summaries. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3832783.3834594



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3834594

1 Introduction

Programming tutorial videos on YouTube are one of the primary self-study resources for students and professionals learning to code [6, 7, 9]. A typical introductory tutorial runs 10–15 minutes, and learners often watch a sequence of such videos to build a complete understanding of a topic. The length of these videos, while pedagogically motivated, makes it expensive to revisit specific concepts or to triage among candidate tutorials on the same topic.

AI-based video summarization has emerged as a way to compress tutorial content into shorter, easier-to-review forms. Existing studies of these systems, however, evaluate their output primarily through similarity-based metrics such as ROUGE or BERTScore, which measure surface overlap between the summary and the source. These metrics tell us whether a summary preserves the words of the original, not whether it preserves what a learner needs to take away. In programming education in particular, the value of a summary depends on whether it preserves *step ordering*, *technical correctness*, and *conceptual completeness*, none of which is captured by lexical overlap.

A handful of recent studies have begun to evaluate AI-generated educational content from a learning-centered perspective [3, 10], but they target generic educational videos rather than programming tutorials, and none of them release a reusable benchmark or tooling that other researchers can build on. As a result, every new summarization tool, prompt design, or model release requires a fresh evaluation setup, and the field has no shared baseline.

Tool overview. We present VidSumEval, a web platform and benchmark for evaluating AI-generated summaries of programming tutorial videos. The platform combines four ingredients: (i) a curated corpus of 10 programming tutorial videos covering C, Java, and Python; (ii) paired AI-generated summaries from two commercial engines (VEED and NotebookLM); (iii) auto-generated comprehension quizzes for each video; and (iv) human quality ratings collected through a within-subject participant study. Users can interact with the platform in two complementary modes: a *browse* mode, in which they explore the benchmark and compare summaries side by side, and a *submit* mode, in which they search YouTube or enter a URL to submit a candidate tutorial for offline review.

Contributions. This paper makes the following contributions:

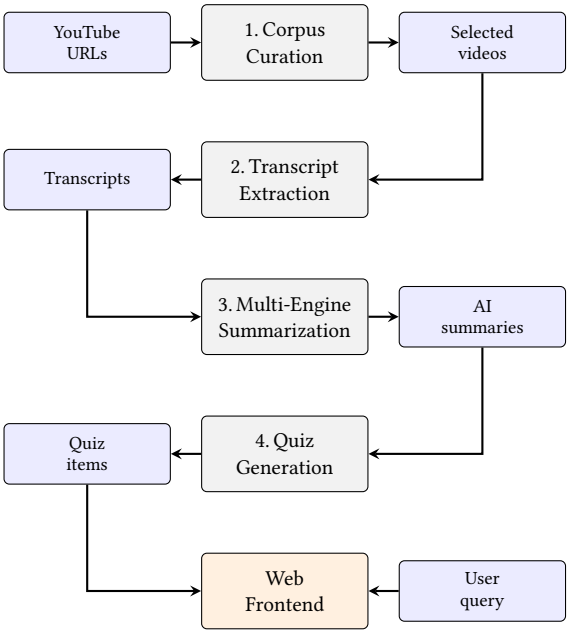


Figure 1: The VidSumEval pipeline. Stages 1–4 run offline to build the benchmark. The web frontend serves browse and submit queries at runtime.

- A learning-centered evaluation framework for AI-generated programming tutorial summaries, pairing quiz-based comprehension scoring with Likert-scale quality ratings on completeness, clarity, and coherence.
- A benchmark of 10 programming tutorials with 20 paired AI-generated summaries, 50 auto-generated quiz items, and 60 evaluation records from 20 participants.
- A deployed web platform with a browse-and-submit interface and a reusable offline pipeline for extending the benchmark.

2 The VidSumEval Approach

Figure 1 shows the four-stage pipeline behind VidSumEval: corpus construction, transcript extraction, multi-engine summary generation, and auto-quiz generation. The stages run offline to populate the benchmark; the web frontend (Section 3) serves both browse and submit queries at runtime.

2.1 Corpus Curation

The corpus curation stage selects programming tutorial videos that are suitable as summarization targets. We started from a candidate pool of 20 YouTube tutorials drawn from established educational channels (e.g., Telusko, Bro Code) and applied three inclusion criteria: (i) clear and structured presentation, (ii) medium difficulty appropriate for university-level learners, and (iii) the presence of both conceptual explanation and practical coding. The final corpus contains 10 videos covering C, Java, and Python, with durations between 10 and 15 minutes. This duration range was selected to provide sufficient instructional content for meaningful summarization while keeping participant sessions manageable and reducing fatigue during the user study. The curation rationale and the list of rejected candidates are recorded in the public release so that future work can extend the corpus consistently.

2.2 Transcript Extraction

The transcript extraction stage converts the spoken narration of each video into clean text using the Eightify transcription service. Transcripts are stored verbatim and serve as the input to the multi-engine summarization stage; they are also used as the source for quiz generation, so that quiz questions test understanding of content the summarizer was given rather than visual content the summarizer never saw.

2.3 Multi-Engine Summary Generation

Each transcript is provided as input to two commercial engines, VEED and NotebookLM, using a single shared prompt that asks for a one-to-three-minute educational summary. The same prompt is used for both engines so that observed differences can be attributed to the engine rather than to prompt variation. For each (video, engine) pair, up to three generation attempts are made if the initial output exceeds the target duration or contains an obvious quality issue (e.g., truncated explanation), and the best of the three attempts is retained. We selected VEED and NotebookLM after pilot testing several candidate engines, retaining the two that produced reliably complete summaries on the candidate corpus; a broader multi-engine comparison is left for future work.

2.4 Auto-Quiz Generation

For each video, a quiz of five multiple-choice questions is generated automatically by prompting Gemini with the original transcript and an instruction template that requires medium-difficulty conceptual questions, including at least two questions that test reasoning or practical application rather than memorization. Quiz items are bound to the source video, not the summary, so that a single quiz can be reused to score participants who watched any of the summaries derived from that video. All generated quiz items, together with the prompt used to produce them, are stored in the benchmark.

3 Tool Walkthrough

The VidSumEval web frontend exposes the benchmark through two complementary modes. Figure 2 shows the browse mode and Figure 3 shows the submit mode.

Browse mode. A user who wants to compare how different AI engines summarize an existing programming tutorial can browse the benchmark of ten original videos. The browse page supports search by title, video code, channel, language, and topic. Selecting a tutorial opens a details page that embeds the original YouTube video and shows the VEED and NotebookLM summaries side by side, together with quiz items and evaluation metrics such as average quiz score, difference score, completeness, clarity, and coherence.

Submit mode. A user who wants to suggest a tutorial that is not yet in the benchmark can search YouTube for candidate programming tutorials or enter a YouTube URL manually. Search results are filtered to prioritize 10–15 minute programming tutorial videos.

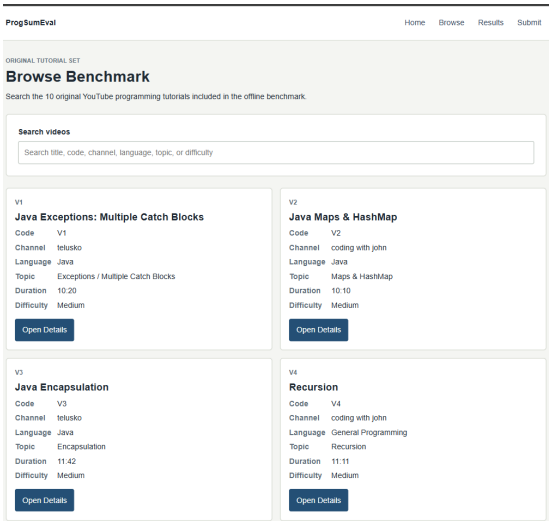


Figure 2: Browse mode. The user filters tutorials by topic, language, or engine; selecting a tutorial opens a side-by-side comparison panel showing both engine summaries with their quality ratings.

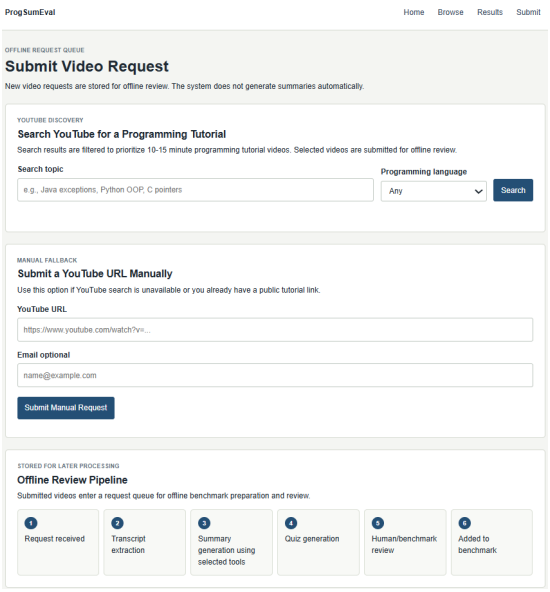


Figure 3: Submit mode. The user searches YouTube or enters a YouTube URL manually. The selected video is stored as a pending request for offline review, and the page explains the benchmark pipeline used before approved videos are added to the dataset.

Selecting or submitting a video stores a pending request for offline review. The page also explains the offline processing pipeline used before an approved video can be added to the benchmark; no summaries or quiz items are generated at runtime.

Table 1: Summary of the VidSumEval benchmark.

Property	Value
Total tutorials	10
Programming languages	3 (C, Java, Python)
Duration range (min)	10–15
Mean / median duration (min)	11.02 / 11.06
Summarization engines	2 (VEED, NotebookLM)
Total summaries	20
Quiz items per video	5
Total quiz items	50
Participants	20
Total evaluation records	60

Implementation. The platform uses Node.js and Express, a SQLite database, and a plain frontend built with HTML, CSS, and JavaScript.

4 Validation

We validate the benchmark with a within-subject study that measures two complementary outcomes. Quiz scores evaluate short-term learning effectiveness and comprehension, whereas Likert-scale ratings evaluate the perceived completeness, clarity, and coherence of the generated summaries.

Benchmark. Table 1 summarizes the dataset distributed with the platform. Each video in the benchmark is paired with two AI-generated summaries, one per engine, and a five-item auto-generated comprehension quiz.

Procedure. Each participant completed one of 10 rotating evaluation forms. Each form contained one original tutorial and two summaries from two different topics, one generated by VEED and one generated by NotebookLM. This rotation ensured that each engine was evaluated across multiple programming topics, that each participant served as their own learning baseline, and that no participant saw the same content twice. After watching each video or summary, the participant completed the corresponding five-item quiz. After watching each summarized video, the participant additionally rated the summary on three quality dimensions: completeness, clarity, and coherence.

For example, one evaluation form may assign an original tutorial on Topic A together with a VEED summary on Topic B and a NotebookLM summary on Topic C. The remaining forms rotate these assignments so that every engine is evaluated across different programming topics while preventing participants from viewing the same tutorial more than once.

Ethics. Participation was voluntary, and participants were informed about the study purpose and the use of their responses before taking part. Responses were collected without directly identifying information, and no sensitive personal data were collected. The study was conducted as a university course research project and did not undergo formal institutional ethics review.

Metrics. We use two complementary metrics:

- **Average Difference Score.** For each participant and evaluation form, we compute $D = S - O$, where S is the participant's quiz score after watching a summarized video and O is the same participant's quiz score after watching the original video assigned in the same form. Because the form uses

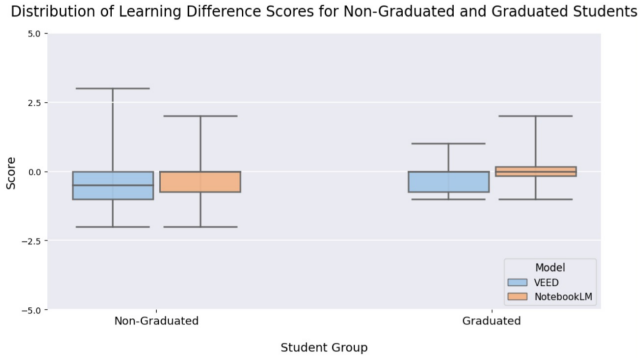


Figure 4: Comparison of learning effectiveness difference scores between undergraduate and graduate participants for VEED and NotebookLM summaries.

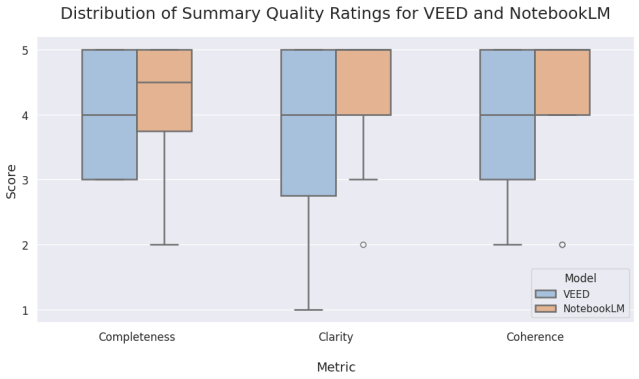


Figure 5: Participant ratings for completeness, clarity, and coherence of VEED and NotebookLM summaries using a five-point Likert scale.

different topics for the original and summarized videos, O serves as a within-participant baseline rather than a same-topic comparison. A value of D near zero indicates that the summary performed close to the original-video baseline; a substantially negative value indicates lower quiz performance after the summary. For example, if a participant scores 4 out of 5 after watching a summarized video and 5 out of 5 after watching the original-video baseline, then the resulting difference score is $D = 4 - 5 = -1$, indicating one fewer correct answer after viewing the summary.

- **Summary Quality Ratings.** Participants rate each summary on *completeness*, *clarity*, and *coherence* using a five-point Likert scale (1 = very poor, 5 = excellent). We report the mean rating per dimension and per engine.

Results.

Figure 4 shows the distribution of learning effectiveness difference scores for undergraduate and graduate participants across VEED and NotebookLM summaries. Across all 20 participants, the mean difference score was $D = -0.200$ for VEED and $D = 0.000$ for NotebookLM, indicating that NotebookLM summaries matched original-video quiz performance on average, while VEED summaries showed a small drop of roughly one-fifth of a quiz point. In both groups, the distributions were generally concentrated within a narrow range, suggesting that summarized videos preserved a substantial portion of the comprehension achieved using the original tutorials. Undergraduate participants exhibited greater variability across both models (SD = 1.337 for VEED and 1.100 for NotebookLM), while graduate participants showed more stable outcomes (SD = 0.738 and 0.876, respectively). NotebookLM generally produced more consistent distributions with fewer low-score deviations compared to VEED.

Figure 5 presents participant ratings for completeness, clarity, and coherence. Both models achieved high completeness ratings (VEED = 4.100, NotebookLM = 4.200), indicating that participants perceived the summaries as sufficiently informative. However, NotebookLM consistently received higher ratings for clarity (4.400 vs. 3.650) and coherence (4.500 vs. 3.950) with lower variability, suggesting that its summaries were easier to follow and more consistently organized.

Overall, the results provide initial evidence that AI-generated programming video summaries can reduce tutorial viewing time while maintaining short-term learner comprehension close to the original-video baseline within the scope of this study.

5 Related Tools

VidSumEval sits between two prior families of tools. The first is *programming-video knowledge tools*, which extract structured artifacts from tutorial videos rather than evaluating their summaries. CodeTube [9] indexes fragments of programming screencasts; ActionNet [11] recognizes developer workflows from screencast frames; Guigle [2] indexes Android GUI screens for visual search. These tools target different artifacts and do not address summary evaluation.

The second family is *AI summarization evaluation*, which has historically relied on similarity-based metrics [4, 5] and, more recently, on learner-perception studies for general educational videos [3, 8, 10]. VidSumEval differs from both lines by combining (i) a focus on programming tutorials, where step ordering and technical correctness matter, with (ii) a learning-centered evaluation framework that pairs comprehension quizzes with quality ratings, and (iii) a reusable web platform and benchmark, rather than a one-off study.

6 Limitations

The current benchmark represents an initial step toward learning-centered evaluation of AI-generated programming video summaries. The benchmark includes only 10 tutorials, two commercial summarization engines, and a within-subject study involving 20 participants, which limits the statistical power and generalizability of the findings. In addition, the evaluation measures short-term comprehension using quiz performance rather than long-term knowledge retention. The comprehension quizzes were automatically generated using a standardized prompt and are publicly available in the Zenodo dataset; however, they were not independently validated by programming education experts, which may affect the reliability of the learning assessment. Future releases of VidSumEval will

expand the benchmark with additional tutorials, include more summarization models, investigate longer-term educational outcomes, and incorporate expert validation of the generated quiz items.

7 Conclusion and Future Work

We presented VidSumEval, a web platform and benchmark for evaluating AI-generated summaries of programming tutorial videos from a learning-centered perspective. The current release includes a pre-evaluated benchmark of 10 tutorials with paired summaries from two summarization engines and a within-subject study reporting both quiz-based learning effectiveness and Likert-scale quality ratings. Planned extensions include adding additional summarization engines, such as open-source LLMs and recent multimodal models, expanding the corpus to cover more languages and difficulty levels, and adding longer-term retention measurement to complement the current short-term comprehension quizzes.

Acknowledgments

OpenAI's ChatGPT was used to assist with language editing, manuscript restructuring, software development, and documentation. The authors reviewed and verified the final manuscript, code, data, and references and take full responsibility for their accuracy and integrity.

8 Data Availability Statement

The VidSumEval source code, benchmark data, quiz items, evaluation records, prompts, and reproduction instructions are archived on Zenodo [1]. The archived record also links to the deployed web platform and screencast.

References

[1] Osama AlGhamdi, Ameer AlGhamdi, Faisal Alorabi, Mohammad D. Alahmadi, Abdulmalik Alzahrani, and Abdullah Almazroui. 2026. *VidSumEval Benchmark Dataset.* doi:10.5281/zenodo.21763079

[2] Carlos Bernal-Cárdenas, Kevin Moran, Michele Tufano, Zichang Liu, Linyong Nan, Zhehan Shi, and Denys Poshyvanyk. 2019. Guigle: A GUI Search Engine for Android Apps. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion).* IEEE, Montreal, QC, Canada, 71–74. doi:10.1109/ICSE-Companion.2019.00041

[3] Hannah Gonzalez, Jiening Li, Helen Jin, Jiaxuan Ren, Hongyu Zhang, Ayotomiwa Akinyele, Adrian Wang, Eleni Miltsakaki, Ryan Baker, and Chris Callison-Burch. 2023. Automatically Generated Summaries of Video Lectures May Enhance Students' Learning Experience. In *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023).* Association for Computational Linguistics, Toronto, Canada, 382–393. doi:10.18653/v1/2023.bea-1.31

[4] Hongxi Li, Yubo Zhu, Zirui Shang, Ziyi Wang, and Xinxiao Wu. 2026. A Comprehensive Survey on Video Summarization: Challenges and Advances. *IEEE Transactions on Circuits and Systems for Video Technology* 36, 1 (Jan. 2026), 1216–1233. doi:10.1109/TCSVT.2025.3596006

[5] J. Lin, H. Hua, M. Chen, Y. Li, J. Hsiao, C. Ho, and J. Luo. 2023. VideoXum: Cross-Modal Video Summarization. *IEEE Trans. Multimedia* 26 (2023), 5548–5560. doi:10.1109/TMM.2023.3335875

[6] Laura MacLeod, Andreas Bergen, and Margaret-Anne Storey. 2017. Documenting and Sharing Software Knowledge Using Screencasts. *Empirical Software Engineering* 22, 3 (2017), 1478–1507. doi:10.1007/s10664-017-9501-9

[7] Laura MacLeod, Margaret-Anne D. Storey, and Andreas Bergen. 2015. Code, Camera, Action: How Software Developers Document and Share Program Knowledge Using YouTube. In *2015 IEEE 23rd International Conference on Program Comprehension (ICPC).* IEEE, Florence, Italy, 104–114. doi:10.1109/ICPC.2015.19

[8] E. Navarrete, A. Nehring, S. Schanze, R. Ewerth, and A. Hoppe. 2025. A Review of Video-Based Learning Research. *Int. J. Artif. Intell. Educ.* 35, 4 (2025), 1631–1694. doi:10.1007/s40593-025-00481-x

[9] Luca Ponzanelli, Gabriele Bavota, Andrea Mocci, Massimiliano Di Penta, Rocco Oliveto, Barbara Russo, Sonia Haiduc, and Michele Lanza. 2016. CodeTube: Extracting Relevant Fragments from Software Development Video Tutorials. In *Proceedings of the 38th International Conference on Software Engineering Companion.* Association for Computing Machinery, New York, NY, USA, 645–648. doi:10.1145/2889160.2889172

[10] T. Xu, Y. Liu, Y. Jin, Y. Qu, J. Bai, W. Zhang, and Y. Zhou. 2025. From Recorded to AI-Generated Instructional Videos. *Br. J. Educ. Technol.* 56, 4 (2025), 1463–1487. doi:10.1111/bjet.13530

[11] Dehai Zhao, Zhenchang Xing, Chunyang Chen, Xin Xia, and Guoqiang Li. 2019. ActionNet: Vision-Based Workflow Action Recognition from Programming Screencasts. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE).* IEEE, Montreal, QC, Canada, 350–361. doi:10.1109/ICSE.2019.00049

Received 2026-05-11; accepted 2026-06-19